

Associative reinforcement learning

Associative reinforcement learning tasks combine facets of stochastic learning automata tasks and supervised learning pattern classification tasks. In associative reinforcement learning tasks, the learning system interacts in a closed loop with its environment.

Associative reinforcement-learning problem is a specific instance of the [reinforcement learning](#) problem whose solution requires generalization and exploration but not temporal credit assignment. In associative reinforcement learning, an action (also called an arm) must be chosen from a fixed set of actions during successive timesteps and from this choice a real-valued reward or payoff results. On each timestep, an input vector is provided that along with the action determines, often probabilistically, the reward. The goal is to maximize the expected long-term reward over a finite or infinite horizon. It is typically assumed that the action choices do not affect the sequence of input vectors. However, even if this assumption is not asserted, learning algorithms are not required to infer or model the relationship between input vectors from one timestep to the next. Requiring a learning algorithm to discover and reason about this underlying process results in the full reinforcement learning problem.

Background

The problem of associative reinforcement learning may be viewed as connecting the problems of [supervised learning](#) or [classification](#), which is more specific, and reinforcement learning, which is more general. Its study is motivated by real-world applications such as choosing which internet advertisements to display based on information about the user or choosing which stock to buy based on current information related to the market. Both problems are distinguished from supervised learning by the absence of labeled training examples to learn from. For instance, in the advertisement problem, the learner is never told which ads would have resulted in the greatest expected reward (in this problem, reward is determined by whether an ad is clicked on or not). In the stock problem, the best choice is never revealed since the choice itself affects the future price of the stocks and therefore the payoff.

The Learning Setting

The learning problem consists of the following core objects:

- An input space X , which is a set of objects (Often a subset of the n -dimension Euclidean space $\mathbb{R}^n$).
- A set of actions or arms A , which is often a finite set of size k .
- A distribution D over X . In some cases, D is allowed to be time-dependent and may be denoted D_t on timestep t for $t = 1, 2, \dots$

A learning sequence proceeds as follows. During each timestep $t = 1, 2, \dots$, an input vector $x_t \in X$ is drawn according to the distribution D and is provided to the algorithm. The algorithm selects an arm at $a_t \in A$. This choice may be stochastic and depend on all previous inputs and rewards observed by the algorithm as well as all previous action choices made by the algorithm for timesteps $t = 1, 2, \dots$. Then, the learner receives a payoff generated according to some unknown stochastic process that depends only on the x_t and a_t . The informal goal is to maximize the expected long-term payoff. Let $\pi : X \rightarrow A$ be any policy that maps input vectors to actions. Let

$$V^\pi(T) := E \left[\sum_{i=1}^T r_i | a_i = \pi(x_i) \text{ for } i = 1, 2, \dots, T \right] \quad (1)$$

denotes the expected total reward over T steps obtained by choosing arms according to policy π . The expectation is taken over any randomness in the generation of input vectors x_i and rewards r_i . The expected regret of a learning algorithm with respect to policy π is defined as $V^\pi(T) - E[\sum_{i=1}^T r_i]$ the expected difference between the return from following policy π and the actual obtained return.

Power of Side Information

Wang, Kulkarni, and Poor (2005) studied the associative reinforcement learning problem from a statistical viewpoint. They considered the setting with two action and analyzed the expected inferior sampling time, which is the number of times that the lesser action, in terms of expected reward, is selected. The function mapping input vectors to conditional reward distributions belongs to a known parameterized class of functions, with the true parameters being unknown. They show that, under some mild conditions, an algorithm can achieve finite expected inferior sampling time. This demonstrates the power provided by the input vectors (also called side observations or side information), because such a result is not possible in the standard multi-armed bandit problem, which corresponds to the associative reinforcement-learning problem without input vectors x_i . Intuitively, this type of result is possible when the side information can be used to infer the payoff function of the optimal action.

Linear Payoff Functions

In its most general setting, the associative reinforcement learning problem is intractable. One way to rectify this problem is to assume that the payoff function is described by a linear system. For instance, Abe and Long (1999) and Auer (2002) consider a model where during each timestep t , there is a vector $z_{i,t}$ associated with each arm i . The expected payoff of pulling arm i on this timestep is given by $\theta^T z_{i,t}$ where θ is an unknown parameter vector and

θ^T denotes the transpose of f . This framework maps to the framework described above by taking $x_t = (z_{t,1}, z_{t,2}, \dots, z_{t,k})$. They assume a time-dependent distribution D and focus on obtaining bounds on the regret against the optimal policy. Assuming that all rewards lie in the interval $[0,1]$, the worst possible regret of any learning algorithm is linear. When considering only the number of timesteps T , Auer (2002) shows that a regret (with respect to the optimal policy) of $O(\sqrt{T} \ln(T))$ can be obtained.

PAC Associative Reinforcement Learning

The previously mentioned works analyze the growth rate of the regret of a learning algorithm with respect to the optimal policy. Another way to approach the problem is to allow the learner some number of timesteps of exploration. After the exploration trials, the algorithm is required to output a policy. More specifically, given inputs $0 < \epsilon < 1$ and $0 < \delta < 1$, the algorithm is required to output an ϵ -optimal policy with probability at least $1 - \delta$. This type of analysis is based on the work by Valiant (1984), and learning algorithms satisfying the above condition are termed probably approximately correct (PAC).

Motivated by the work of Kaelbling (1994), Fiechter (1995) developed a PAC algorithm when the true payoff function can be described by a decision list over the action and input vector. Building on both works, Strehl, Mesterharm, Littman, and Hirsh (2006) showed that a class of associative reinforcement learning problems can be solved efficiently, in a PAC sense, when given a learning algorithm for efficiently solving classification problems.

Deep reinforcement learning

Deep Reinforcement Learning (DRL), a very fast-moving field, is the combination of Reinforcement Learning and Deep Learning. It is also the most trending type of Machine Learning because it can solve a wide range of complex decision-making tasks that were previously out of reach for a machine to solve real-world problems with human-like intelligence.

In the past few years, Deep Learning techniques have become very popular. Deep Reinforcement Learning is the combination of Reinforcement Learning with Deep Learning techniques to solve challenging sequential decision-making problems. The use of deep learning is most useful in problems with high-dimensional state space. This means that with deep learning, Reinforcement Learning is able to solve more complicated tasks with lower prior knowledge because of its ability to learn different levels of abstractions from data. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. This makes it possible for machines to mimic some human problem-solving capabilities, even in high-dimensional space, which only a few years ago was difficult to conceive.

Deep Reinforcement Learning is one of the most exciting fields in Artificial Intelligence. It's marrying the power and the ability of deep neural networks to represent and comprehend the world with the ability to act on that understanding.

Deep RL is a type of Machine Learning where an agent learns how to behave in an environment by performing actions and seeing the results.

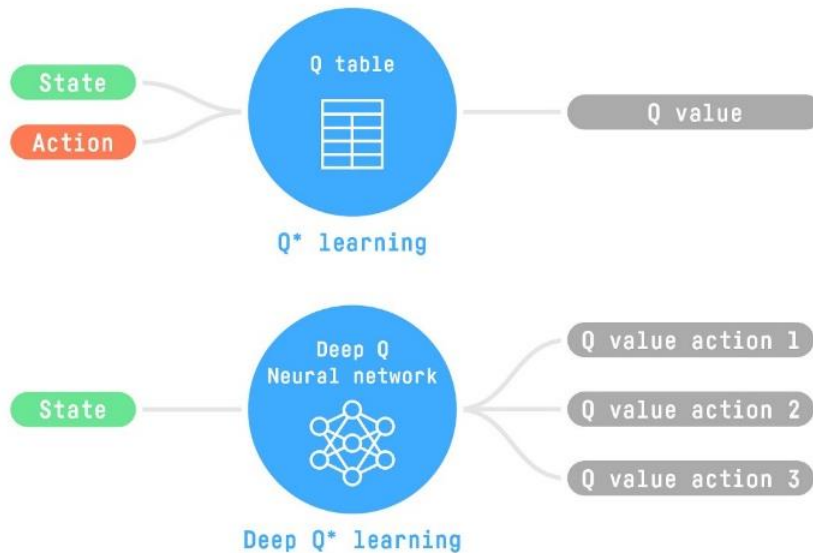
Deep Reinforcement Learning introduces deep neural networks to solve Reinforcement Learning problems — hence the name “deep”.

For instance, we’ll learn about two value-based algorithms:

- Q-Learning (classic Reinforcement Learning) and
- Deep Q-Learning.

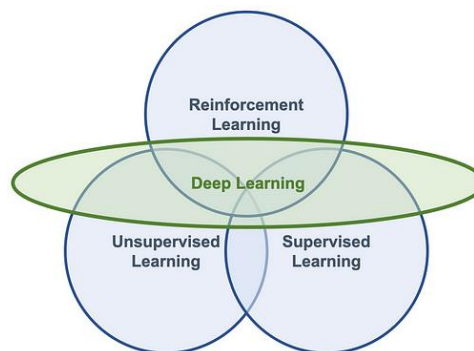
We’ll see the difference is that, in the first approach, we use a traditional algorithm to create a Q table that helps us find what action to take for each state.

In the second approach, we will use a Neural Network (to approximate the Q value).



Remember that Deep Learning algorithms are based on artificial neural networks, whose algorithmic structures allow models composed of multiple processing layers to learn data representations with various abstraction levels.

DL is not a separate ML branch. DL is a collection of techniques and methods for using neural networks to solve ML tasks, either Supervised Learning, Unsupervised Learning, or Reinforcement Learning. We can represent it graphically in following figure.



Visual relationship between Deep Learning and Machine Learning categories.

Deep Learning is one of the best tools that we have today to handle unstructured environments; they can learn from large amounts of data or discover patterns. But this is not decision-making; it is a recognition problem. Reinforcement Learning provides this feature.

Reinforcement Learning can solve the problems using a variety of ML methods and techniques, from decision trees to SVMs, to neural networks. However, in this series, we only use neural networks; this is what the “deep” part of DRL refers to, after all. However, neural networks are not necessarily the best solution to every problem. For instance, neural networks are very data-hungry and challenging to interpret. Still, without doubt, neural networks are at this moment one of the most powerful techniques available, and their performance is often the best.

Applications of Deep Reinforcement Learning

Reinforcement Learning has evolved rapidly over the past few years with a wide range of applications. One of the primary reasons for this evolution is the combination of Reinforcement Learning and Deep Learning. This is why we focus this series on presenting the basic state-of-the-art Deep Reinforcement Learning algorithms (DRL).

Real-life applications of DRL

The media has tended to focus on applications where DRL defeat humans at games, with examples as I mentioned at the beginning of this post: AlphaGo defeated the best professional human player in the game of Go; AlphaStar beat professional players at the game of StarCraft II; OpenAI's Dota-2-playing bot beat the world champions in an e-sports game.

Fortunately, there are many real-life applications of DRL. One of the well known is in the area of driverless cars. In manufacturing, intelligent robots are trained using DRL to place objects in the right position, reducing labor costs, and increasing productivity. Another popular application of RL is dynamic pricing that allows changing the price of products based on demand and supply. Also, in a recommendation system, RL is used to build a recommendation system where the user's behavior continually changes.

In today's business activities, DRL is used extensively in supply chain management, demand forecasting, inventory management, handling warehouse operations, etc. DRL is also widely used in financial portfolio management, predicting, and trading in commercial transaction markets. DRL has been commonly used in several Natural Language Processing (NLP) tasks, such as abstractive text summarization, chatbots, etc.

Many recent research papers suggest applications of DRL in healthcare, education systems, smart cities, among many others. In summary, no business sector is left untouched by DRL. *----- Need to modify with following list of applications-----*****

Some prominent projects used deep Reinforcement Learning in games with results that are far beyond what is humanly possible. Deep RL techniques have demonstrated their ability to tackle a wide range of problems that were previously unsolved. Deep RL has achieved human-level or superhuman performance for many two-player or even multi-player games. Such achievements with popular games are significant because they show the potential of deep Reinforcement Learning in a variety of complex and diverse tasks that are based on high-dimensional inputs. With games, we have good or even perfect simulators, and can easily generate unlimited data.

- Atari 2600 games: Machines achieved superhuman-level performance in playing Atari games.
- Go: Mastering the game of Go with deep neural networks.
- Poker: AI is able to beat professional poker players in the game of heads-up no-limit Texas hold'em.
- Quake III: An agent achieved human-level performance in a 3D multiplayer first-person video game, using only pixels and game points as input.
- Dota 2: An AI agent learned to play Dota 2 by playing over 10,000 years of games against itself (OpenAI Five).
- StarCraft II: An agent was able to learn how to play StarCraft II a 99% win-rate, using only 1.08 hours on a single commercial machine.

Those achievements set the basis for the development of real-world deep reinforcement learning applications:

- Robot control: Robotics is a classical application area for reinforcement learning. Robust adversarial reinforcement learning is applied as an agent operates in the presence of a destabilizing adversary that applies disturbance forces to the system. The machine is trained to learn an optimal destabilization policy. AI-powered robots have a wide range of applications, e.g. in manufacturing, supply chain automation, healthcare, and many more.
- Self-driving cars: Deep Reinforcement Learning is prominently used with autonomous driving. Autonomous driving scenarios involve interacting agents and require negotiation and dynamic decision-making which suits Reinforcement Learning.
- Healthcare: In the medical field, Artificial Intelligence (AI) has enabled the development of advanced intelligent systems able to learn about clinical treatments, provide clinical decision support, and discover new medical knowledge from the huge amount of data collected. Reinforcement Learning enabled advances such as personalized medicine that is used to systematically optimize patient health care, in particular, for chronic conditions and cancers using individual patient information.

- Other: In terms of applications, many areas are likely to be impacted by the possibilities brought by deep Reinforcement Learning, such as finance, business management, marketing, resource management, education, smart grids, transportation, science, engineering, or art. In fact, Deep RL systems are already in production environments. For example, Facebook uses Deep Reinforcement Learning for pushing notifications and for faster video loading with smart prefetching.

Challenges of Deep Reinforcement Learning

Multiple challenges arise in applying Deep Reinforcement Learning algorithms. In general, it is difficult to explore the environment efficiently or to generalize good behavior in a slightly different context. Therefore, multiple algorithms have been proposed for the Deep Reinforcement Learning framework, depending on a variety of settings of the sequential decision-making tasks.

Many challenges appear when moving from a simulated setting to solving real-world problems.

- Limited freedom of the agent: In practice, even in the case where the task is well-defined (with explicit reward functions), a fundamental difficulty lies in the fact that it is often not possible to let the agent interact freely and sufficiently in the actual environment, due to safety, cost or time constraints.
- Reality gap: There may be situations, where the agent is not able to interact with the true environment but only with an inaccurate simulation of it. The reality gap describes the difference between the learning simulation and the effective real-world domain.
- Limited observations: For some cases, the acquisition of new observations may not be possible anymore (e.g., the batch setting). Such scenarios occur, for example, in medical trials or tasks with dependence on weather conditions or trading markets such as stock markets.

How those challenges can be addressed:

- Simulation: For many cases, a solution is the development of a simulator that is as accurate as possible.
- Algorithm Design: The design of the learning algorithms and their level of generalization has a great impact.
- Transfer Learning: Transfer learning is a crucial technique to utilize external expertise from other tasks to benefit the learning process of the target task.

Reinforcement Learning and Computer Vision

Computer Vision is about how computers gain understanding from digital images and video streams. Computer Vision has been making rapid progress recently, and deep learning plays an important role.

Reinforcement learning is an effective tool for many computer vision problems, like image classification, object detection, face detection, captioning, and more. Reinforcement Learning is an important ingredient for interactive perception, where perception and interaction with the environment would be helpful to each other. This includes tasks like object segmentation, articulation model estimation, object dynamics learning, haptic property estimation, object recognition or categorization, multimodal object model learning, object pose estimation, grasp planning, and manipulation skill learning.



Object detection with detected classes – A real-time application

More topics of applying Deep Reinforcement Learning to computer vision tasks, such as:

- Semantic parsing of large-scale 3D point clouds for indoor scene understanding

- Teaching a machine to read maps with deep reinforcement learning
- Image-based data augmentation tasks deep reinforcement learning
- View Planning, to generate a sequence of viewpoints that are capable of sensing all accessible areas of a given object represented as a 3D model